Knowledge Representation for Cross-Border Payroll Tax Calculation and Annual Tax Form Generation

AIMLZG557 · Artificial and Computational Intelligence · Situated Learning Assignment · Question 2 (Individual Submission)

Field	Entry
Name	Aditya Saraswat
BITS ID	2025AG05058
Industry / Domain	Human Capital Management / Payroll Technology – US and Canada Payroll
Organisation Type	Enterprise payroll and HR technology organisation
Workplace process	Cross-border payroll tax assessment, validation and annual tax form routing

Scope note. This assignment uses synthetic payroll records to demonstrate logical knowledge representation. It models tax-component routing, payroll validation and annual-form eligibility; it does not calculate statutory tax amounts. Actual amounts require current jurisdiction- and tax-year-specific rules, tables, elections, taxable-earning definitions and YTD balances.

B.1 Submission Form

The selected workplace process is US and Canadian payroll processing, focusing on tax calculation readiness, statutory tax-component assessment and annual tax form generation. The knowledge base determines which calculation paths apply, whether payroll can be finalised and which year-end reporting workflow should be initiated.

B.2 Anchor

Workplace artifact: Cross-Border Payroll Tax Calculation and Annual Tax Form Generation Workflow.

Why it matters. Payroll must calculate gross-to-net pay accurately while applying the correct country, state, province, territory and local rules. Incorrect worker classification, work/residence location, tax elections, taxable-benefit classification or YTD balances can cause incorrect withholding or annual statements.

Process flow. (1) Maintain worker classification, work/residence locations, tax elections and effective dates. (2) Collect earnings, hours, taxable benefits, deductions and YTD balances. (3) Determine applicable jurisdictions. (4) Apply current tax tables and contribution parameters. (5) Validate results and route exceptions. (6) Accumulate finalised payroll results. (7) Route eligible records to W-2, relevant 1099 review, T4 or other applicable information-slip workflows; validate and correct forms when required.

People involved. Employees/contractors maintain tax information; payroll operations review inputs and exceptions; tax specialists maintain jurisdictional rules; engineering/product teams maintain calculation and form services; compliance/legal teams interpret reporting obligations.

Sources of facts. Worker master data, work/residence locations, tax elections, payroll inputs, taxable-benefit configuration, YTD accumulators, statutory parameter tables, calculation results, validation logs, year-end accumulators and form-generation records.

Illustrative records. Six synthetic cases: US employees U101 and U102; US contractors C301 and C302; Canadian employees U201 and U202. No real employee or payroll data is used.

B.3 Apply

The process is represented through 15 business rules, propositional and predicate logic, a knowledge base containing 80 explicit ground facts and 15 executable inference-rule schemas. Forward chaining derives conclusions from facts; backward chaining explains selected queries from their target conclusions.

B.3.1 Business rules

Rule	Business rule
R1	A payroll case needs an identified jurisdiction and tax year before it is in scope.
R2	Worker classification as employee or contractor must be known before calculation/reporting routing.
R3	Calculation readiness requires scope, classification, complete inputs and a valid tax profile.
R4	The taxable earnings/benefits base must be ready before tax components are assessed.
R5	A US employee with a validated taxable base enters the federal withholding path.
R6	A US employee enters Social Security assessment, subject to annual wage base and YTD rules.
R7	A US employee enters Medicare assessment, including additional Medicare checks where relevant.
R8	US state/local tax is assessed when work/residence facts establish an obligation.
R9	A Canadian employee enters federal and applicable provincial/territorial income-tax withholding.
R10	A Canadian employee enters CPP/QPP and EI assessment, subject to exemptions and annual limits.
R11	Payroll is finalised only after calculation and successful validation.
R12	A finalised US employee with reportable wages and year-end readiness is routed to W-2 generation.
R13	A finalised US contractor with reportable payments and year-end readiness enters 1099 type/obligation review.
R14	A finalised Canadian employee with reportable employment income and year-end readiness is routed to T4.
R15	A post-issuance adjustment changing annual-form data initiates the applicable corrected-form workflow.

B.3.2 Propositional logic

Each proposition describes one payroll case. ∧ = AND, ∨ = OR, ¬ = NOT, → = implies.

Symbol	Meaning	Symbol	Meaning
J / Y	Jurisdiction / tax year identified	E / C	Employee / contractor
I / V	Inputs complete / profile valid	READY	Calculation prerequisites met
TB	Taxable base ready	US / CA	US / Canada case
FED	Federal withholding path	SS / MED	Social Security / Medicare
ST	State/local obligation	CIT	Canadian income-tax path
CPP / EI	Canadian contribution assessment	CALC	Tax calculation performed
PASS / FINAL	Validation passed / payroll finalised	RW / RC	Reportable wages / contractor payment
REPY	Reportable Canadian employment income	YE	Year-end ready
W2 / N1099 / T4	Annual-form route	ISS / ADJ / CHG	Issued / adjustment / form data changed
CORR	Corrected form required	CLASSIFIED	Worker class known

Rule	Propositional formula
R1	J ∧ Y → SCOPE
R2	(E ∨ C) → CLASSIFIED
R3	SCOPE ∧ CLASSIFIED ∧ I ∧ V → READY
R4	READY ∧ TB → TAXBASE
R5	US ∧ E ∧ TAXBASE → FED
R6	US ∧ E ∧ TAXBASE → SS
R7	US ∧ E ∧ TAXBASE → MED
R8	US ∧ E ∧ ST → STATE_LOCAL
R9	CA ∧ E ∧ TAXBASE → CIT
R10	CA ∧ E ∧ TAXBASE → CPP ∧ EI
R11	READY ∧ CALC ∧ PASS → FINAL
R12	US ∧ E ∧ FINAL ∧ RW ∧ YE → W2
R13	US ∧ C ∧ FINAL ∧ RC ∧ YE → N1099
R14	CA ∧ E ∧ FINAL ∧ REPY ∧ YE → T4
R15	ISS ∧ ADJ ∧ CHG → CORR

Interpretation. These rules establish calculation and reporting paths, not tax amounts. The 1099 conclusion is a review route because payment type and current reporting rules determine the specific 1099 form and obligation. A generic contractor label alone is insufficient to determine a 1099 variant.

B.3.3 Predicate logic

Constants U101, U102, C301, C302, U201 and U202 identify synthetic workers; P1–P6 identify payroll cases; 2026 is the illustrative tax year. Variables x = worker, p = payroll case, y = tax year, f = form and a = adjustment.

Predicate	Meaning	Kind
Worker(x), PayrollCase(p)	Worker and payroll case	Fact
Employee(x), Contractor(x)	Worker classification	Fact
WorkerOnCase(x,p)	Worker x is associated with case p	Fact
JurisdictionKnown(p), TaxYearKnown(p,y)	Jurisdiction and tax year identified	Fact
InputsComplete(p), ProfileValid(p)	Inputs/profile complete and valid	Fact
TaxableBaseReady(p), TaxCalculated(p)	Taxable base ready; calculation performed	Fact
ValidationPassed(p), ValidationFailed(p)	Validation outcome	Fact
USCase(p), CanadaCase(p)	Payroll country	Fact
FederalWithholding(p), SocialSecurityAssessment(p), MedicareAssessment(p)	US tax-component paths	Derived
StateLocalObligation(p), StateLocalTaxPath(p)	US state/local obligation and path	Fact/Derived
CanadianIncomeTaxPath(p), CPPQPPAssessment(p), EIAssessment(p)	Canadian tax-component paths	Derived
CalculationReady(p), TaxBaseValidated(p), Finalised(p)	Readiness, tax base validation and finalisation	Derived
ReportableWages(x,y), ReportableContractorPay(x,y)	US reporting facts	Fact
ReportableEmploymentIncome(x,y), YearEndReady(x,y)	Canadian reporting/year-end facts	Fact
W2Required(x,y), Form1099Review(x,y), T4Required(x,y)	Annual-form routes	Derived
FormIssued(f,x,y), AdjustmentRecorded(a,x,y), FormDataChanged(a,x,y)	Form and adjustment facts	Fact
CorrectedFormRequired(f,x,y)	Correction workflow required	Derived

Rules in predicate logic

P1 $\forall p \forall y\ [(\text{JurisdictionKnown}(p) \wedge \text{TaxYearKnown}(p,y)) \rightarrow \text{ScopeReady}(p)]$

P2 $\forall x\ [(\text{Employee}(x) \vee \text{Contractor}(x)) \rightarrow \text{ClassificationKnown}(x)]$

P3 $\forall p \forall x\ [\text{ScopeReady}(p) \wedge \text{ClassificationKnown}(x) \wedge \text{WorkerOnCase}(x,p) \wedge \text{InputsComplete}(p) \wedge \text{ProfileValid}(p) \rightarrow \text{CalculationReady}(p)]$

P4 $\forall p\ [\text{CalculationReady}(p) \wedge \text{TaxableBaseReady}(p) \rightarrow \text{TaxBaseValidated}(p)]$

P5 $\forall p \forall x\ [(\text{USCase}(p) \wedge \text{Employee}(x) \wedge \text{WorkerOnCase}(x,p) \wedge \text{TaxBaseValidated}(p)) \rightarrow \text{FederalWithholding}(p)]$

P6 $\forall p \forall x\ [(\text{USCase}(p) \wedge \text{Employee}(x) \wedge \text{WorkerOnCase}(x,p) \wedge \text{TaxBaseValidated}(p)) \rightarrow \text{SocialSecurityAssessment}(p)]$

P7 $\forall p \forall x\ [(\text{USCase}(p) \wedge \text{Employee}(x) \wedge \text{WorkerOnCase}(x,p) \wedge \text{TaxBaseValidated}(p)) \rightarrow \text{MedicareAssessment}(p)]$

P8 $\forall p \forall x\ [(\text{USCase}(p) \wedge \text{Employee}(x) \wedge \text{WorkerOnCase}(x,p) \wedge \text{StateLocalObligation}(p)) \rightarrow \text{StateLocalTaxPath}(p)]$

P9 $\forall p \forall x\ [(\text{CanadaCase}(p) \wedge \text{Employee}(x) \wedge \text{WorkerOnCase}(x,p) \wedge \text{TaxBaseValidated}(p)) \rightarrow \text{CanadianIncomeTaxPath}(p)]$

P10 $\forall p \forall x\ [(\text{CanadaCase}(p) \wedge \text{Employee}(x) \wedge \text{WorkerOnCase}(x,p) \wedge \text{TaxBaseValidated}(p)) \rightarrow \text{CPPQPPAssessment}(p) \wedge \text{EIAssessment}(p)]$

P11 $\forall p\ [\text{CalculationReady}(p) \wedge \text{TaxCalculated}(p) \wedge \text{ValidationPassed}(p) \rightarrow \text{Finalised}(p)]$

P12 $\forall p \forall x \forall y\ [(\text{USCase}(p) \wedge \text{Employee}(x) \wedge \text{WorkerOnCase}(x,p) \wedge \text{Finalised}(p) \wedge \text{ReportableWages}(x,y) \wedge \text{YearEndReady}(x,y)) \rightarrow \text{W2Required}(x,y)]$

P13 $\forall p \forall x \forall y\ [(\text{USCase}(p) \wedge \text{Contractor}(x) \wedge \text{WorkerOnCase}(x,p) \wedge \text{Finalised}(p) \wedge \text{ReportableContractorPay}(x,y) \wedge \text{YearEndReady}(x,y)) \rightarrow \text{Form1099Review}(x,y)]$

P14 $\forall p \forall x \forall y\ [(\text{CanadaCase}(p) \wedge \text{Employee}(x) \wedge \text{WorkerOnCase}(x,p) \wedge \text{Finalised}(p) \wedge \text{ReportableEmploymentIncome}(x,y) \wedge \text{YearEndReady}(x,y)) \rightarrow \text{T4Required}(x,y)]$

P15 $\forall f \forall x \forall y \forall a\ [(\text{FormIssued}(f,x,y) \wedge \text{AdjustmentRecorded}(a,x,y) \wedge \text{FormDataChanged}(a,x,y)) \rightarrow \text{CorrectedFormRequired}(f,x,y)]$

Quantifier use. $\forall$ applies each rule to every matching worker, payroll case and tax year. P12–P14 link worker, case and year-end reporting facts. P15 connects an issued form to a later adjustment. Production rules would further distinguish federal, state, provincial, territorial and local jurisdictions.

B.3.4 Knowledge base

KB = F ∪ R. F contains 80 explicit synthetic ground facts across six cases; R contains 15 inference-rule schemas. Derived facts are excluded from the initial count. The executable uses a closed-world check for the demonstration: a missing prerequisite does not satisfy a rule. In production, missing information should be UNKNOWN and routed for review rather than silently treated as a negative tax fact.

Case	Worker / scenario	Expected result
P1	U101 – US employee; complete inputs, valid profile, taxable base ready, calculation and validation passed; state/local obligation; reportable wages	US tax paths; finalised; W-2
P2	U102 – US employee; payroll validated; W-2 issued; later adjustment changes form data	Finalised; corrected W-2 workflow
P3	C301 – US contractor; calculation and validation passed; reportable contractor payment	Finalised; 1099 type/reporting review
P4	C302 – US contractor; contractor payment explicitly not reportable in this synthetic case	No 1099 route
P5	U201 – Canadian employee; payroll validated; reportable employment income	Canadian tax paths; finalised; T4
P6	U202 – Canadian employee; calculation attempted but validation failed; year-end not ready	Not finalised; exception review

Facts by case

Case	Ground facts represented	Count
P1 / U101	Worker, Employee, USCase, WorkerOnCase, JurisdictionKnown, TaxYearKnown(2026), InputsComplete, ProfileValid, TaxableBaseReady, TaxCalculated, ValidationPassed, StateLocalObligation, ReportableWages, YearEndReady	14
P2 / U102	Worker, Employee, USCase, WorkerOnCase, JurisdictionKnown, TaxYearKnown(2026), InputsComplete, ProfileValid, TaxableBaseReady, TaxCalculated, ValidationPassed, ReportableWages, YearEndReady, FormIssued, AdjustmentRecorded, FormDataChanged	16
P3 / C301	Worker, Contractor, USCase, WorkerOnCase, JurisdictionKnown, TaxYearKnown(2026), InputsComplete, ProfileValid, TaxCalculated, ValidationPassed, ReportableContractorPay, YearEndReady	12
P4 / C302	Worker, Contractor, USCase, WorkerOnCase, JurisdictionKnown, TaxYearKnown(2026), InputsComplete, ProfileValid, TaxCalculated, ValidationPassed, NotReportableContractorPay, YearEndReady	12
P5 / U201	Worker, Employee, CanadaCase, WorkerOnCase, JurisdictionKnown, TaxYearKnown(2026), InputsComplete, ProfileValid, TaxableBaseReady, TaxCalculated, ValidationPassed, ReportableEmploymentIncome, YearEndReady	13
P6 / U202	Worker, Employee, CanadaCase, WorkerOnCase, JurisdictionKnown, TaxYearKnown(2026), InputsComplete, ProfileValid, TaxableBaseReady, TaxCalculated, ValidationFailed, ReportableEmploymentIncome, YearEndNotReady	13
TOTAL	Initial explicit facts; derived facts excluded	80

Configuration assumptions. The year 2026 is illustrative. Statutory rates, brackets, wage bases, contribution limits, provincial/territorial parameters, local tax rules and form schemas are not hardcoded. Tax-component predicates route the case to a separate statutory calculation engine. Contractor classification alone does not determine a specific 1099 form.

B.3.5 Five workplace queries

Query 1 – Does US employee U101 enter the W-2 route, and which tax paths apply? Method: forward chaining. P1 derives ScopeReady(P1); P2 derives ClassificationKnown(U101); P3 derives CalculationReady(P1); P4 derives TaxBaseValidated(P1). P5–P7 derive federal withholding, Social Security and Medicare assessment paths. P8 derives StateLocalTaxPath(P1) because the obligation fact exists. P11 derives Finalised(P1), and P12 derives W2Required(U101, 2026). **Answer: yes.**

Query 2 – Does US contractor C301 enter the 1099 review path? Method: backward chaining from Form1099Review(C301, 2026). P13 requires USCase, Contractor, WorkerOnCase, Finalised, ReportableContractorPay and YearEndReady. All facts are supported; P11 proves Finalised(P3). **Answer: yes.** Route to the correct 1099 type/reporting review; the model does not select a specific 1099 variant.

Query 3 – Is Canadian employee U201 ready for T4, and which tax paths apply? Method: forward chaining. P1–P4 establish calculation readiness and validated taxable base. P9 derives CanadianIncomeTaxPath(P5); P10 derives CPPQPPAssessment(P5) and EIAssessment(P5). P11 derives Finalised(P5), and P14 derives T4Required(U201, 2026). **Answer: yes.**

Query 4 – Can Canadian payroll case P6 for U202 be finalised? Method: backward chaining from Finalised(P6). P11 requires CalculationReady, TaxCalculated and ValidationPassed. The first two are supported, but ValidationFailed(P6) is explicit and ValidationPassed(P6) is absent. **Answer: no.** Do not finalise; route to payroll exception review. Year-end is also explicitly not ready.

Query 5 – Is a corrected W-2 workflow required for U102? Method: forward chaining. FormIssued(W2,U102,2026), AdjustmentRecorded(A1,U102,2026) and FormDataChanged(A1,U102,2026) satisfy P15. **Answer: yes.** Initiate the applicable corrected W-2 process, which may include Form W-2c.

B.4 Analyse

Predicate Logic compared with Propositional Logic. Propositional Logic can describe one payroll case, for example US ∧ E ∧ FINAL ∧ RW ∧ YE → W2. A separate set of propositions would be needed for every worker, case, jurisdiction and tax year. Predicate Logic introduces variables and quantifiers, allowing one rule to apply to every eligible US employee across years. It also represents relationships among workers, payroll cases, adjustments and forms without duplicating rules.

Effectiveness. The knowledge base makes control and routing decisions explicit: it identifies country-specific tax paths, checks calculation prerequisites, prevents finalisation when validation fails and routes eligible workers to annual-form workflows. Proof traces make the conclusions explainable and testable. The same rules can be reused as new workers and cases are added.

Limitations. The implementation does not calculate withholding amounts. Actual calculation requires current federal, state, provincial, territorial and local rules; payroll frequency; tax elections; taxable earnings/benefits; YTD accumulators; annual wage bases; contribution limits; exemptions and effective dates. The model does not fully represent state/local nexus, Quebec-specific treatment, cross-border assignments, tax treaties, retroactive payroll or all contractor payment categories. A generic contractor label cannot determine a 1099 form by itself.

Other risks include stale parameters, incorrect work location, incomplete tax elections, effective-date errors, rounding differences, duplicate inputs, incorrect YTD balances and unreliable interfaces. A knowledge base can explain a wrong answer if its source facts are wrong; data quality and governance are essential.

B.5 Reflect

Practical constraints. (1) Payroll facts are distributed across worker master data, time/earnings systems, tax profile services, accumulators and form services. (2) Tax rules change by year and jurisdiction, so versioning is mandatory. (3) Payroll data requires role-based access, encryption, retention controls and audit logs. (4) Large populations and multiple pay frequencies require scalable processing. (5) Cross-border assignments and unusual payment categories require specialist review. (6) Form corrections and retroactive adjustments must preserve links to original records.

Single most important improvement before deployment. Introduce a governed, versioned jurisdiction-and-tax-year rules/parameter layer, with automated validation and authorised human approval before any tax table or rule becomes active.

This is the priority because a logically correct inference can still produce an incorrect payroll result if it uses an outdated rate, threshold, wage base, exemption, provincial parameter or form schema. Each result should retain the exact rule/table version, source, effective date, input facts, derived outcome and human overrides. Missing or contradictory data should create a Pending/Needs Review state rather than being silently treated as false.

What I learned. Formalising payroll as logic separates the decision to enter a tax path from calculation of the statutory amount. The same inference framework routes US employees, US contractors and Canadian employees differently using jurisdiction and classification facts. A calculation that has run is not necessarily a result that can be finalised; validation is a separate gate.

Conclusion

A rule-based knowledge base can make cross-border payroll more consistent and explainable by formalising calculation readiness, tax-component routing, payroll validation and annual-form workflows. Predicate Logic makes rules reusable across workers, cases and tax years. Production use still requires authoritative version-controlled parameters, reliable data, privacy controls, audit trails and human review.

Appendix A Python implementation

The knowledge base is implemented in Python 3 using only the standard library. **payroll_kb.py** contains 80 synthetic ground facts, all 15 inference rules P1–P15, forward chaining until no new facts are derived, and backward-chaining proof traces for selected queries. It was executed successfully with exit code 0.

Execution: Run payroll_kb.py with the project's Python 3 interpreter.

```
"""
AIMLZG557 - Question 2
Cross-Border Payroll Knowledge Base: US and Canada
Synthetic, explainable rule-based demonstration.



The program demonstrates jurisdiction routing, tax-component eligibility,
payroll validation/finalisation, and annual-form routing. It does NOT calculate
statutory tax amounts or replace the production payroll tax engine.
"""

from collections import defaultdict

YEAR = 2026

# Facts use the form (predicate, argument1, argument2, ...).
# Every record is synthetic; no employee or payroll data is real.
FACTS = {
    # US employee U101 / P1: valid payroll, state/local obligation, W-2 due.
    ("Worker", "U101"), ("Employee", "U101"), ("USCase", "P1"),
    ("WorkerOnCase", "U101", "P1"), ("JurisdictionKnown", "P1"),
    ("TaxYearKnown", "P1", YEAR), ("InputsComplete", "P1"),
    ("ProfileValid", "P1"), ("TaxableBaseReady", "P1"),
    ("TaxCalculated", "P1"), ("ValidationPassed", "P1"),
    ("StateLocalObligation", "P1"), ("ReportableWages", "U101", YEAR),
    ("YearEndReady", "U101", YEAR),

    # US employee U102 / P2: valid payroll and an issued W-2 requiring correction.
    ("Worker", "U102"), ("Employee", "U102"), ("USCase", "P2"),
    ("WorkerOnCase", "U102", "P2"), ("JurisdictionKnown", "P2"),
    ("TaxYearKnown", "P2", YEAR), ("InputsComplete", "P2"),
    ("ProfileValid", "P2"), ("TaxableBaseReady", "P2"),
    ("TaxCalculated", "P2"), ("ValidationPassed", "P2"),
    ("ReportableWages", "U102", YEAR), ("YearEndReady", "U102", YEAR),
    ("FormIssued", "W2", "U102", YEAR),
    ("AdjustmentRecorded", "A1", "U102", YEAR),
    ("FormDataChanged", "A1", "U102", YEAR),

    # US contractor C301 / P3: reportable contractor payment, 1099 review.
    ("Worker", "C301"), ("Contractor", "C301"), ("USCase", "P3"),
    ("WorkerOnCase", "C301", "P3"), ("JurisdictionKnown", "P3"),
    ("TaxYearKnown", "P3", YEAR), ("InputsComplete", "P3"),
    ("ProfileValid", "P3"), ("TaxCalculated", "P3"),
    ("ValidationPassed", "P3"), ("ReportableContractorPay", "C301", YEAR),
    ("YearEndReady", "C301", YEAR),

    # US contractor C302 / P4: payment not reportable under this synthetic case.
    ("Worker", "C302"), ("Contractor", "C302"), ("USCase", "P4"),
    ("WorkerOnCase", "C302", "P4"), ("JurisdictionKnown", "P4"),
    ("TaxYearKnown", "P4", YEAR), ("InputsComplete", "P4"),
    ("ProfileValid", "P4"), ("TaxCalculated", "P4"),
    ("ValidationPassed", "P4"), ("NotReportableContractorPay", "C302", YEAR),
    ("YearEndReady", "C302", YEAR),

    # Canadian employee U201 / P5: valid payroll, T4 due.
    ("Worker", "U201"), ("Employee", "U201"), ("CanadaCase", "P5"),
    ("WorkerOnCase", "U201", "P5"), ("JurisdictionKnown", "P5"),
    ("TaxYearKnown", "P5", YEAR), ("InputsComplete", "P5"),
    ("ProfileValid", "P5"), ("TaxableBaseReady", "P5"),
    ("TaxCalculated", "P5"), ("ValidationPassed", "P5"),
    ("ReportableEmploymentIncome", "U201", YEAR),
    ("YearEndReady", "U201", YEAR),

    # Canadian employee U202 / P6: calculation attempted but validation failed.
    ("Worker", "U202"), ("Employee", "U202"), ("CanadaCase", "P6"),
    ("WorkerOnCase", "U202", "P6"), ("JurisdictionKnown", "P6"),
    ("TaxYearKnown", "P6", YEAR), ("InputsComplete", "P6"),
    ("ProfileValid", "P6"), ("TaxableBaseReady", "P6"),
    ("TaxCalculated", "P6"), ("ValidationFailed", "P6"),
    ("ReportableEmploymentIncome", "U202", YEAR),
    ("YearEndNotReady", "U202", YEAR),
}

RULES = [
    ("P1", "JurisdictionKnown(p) AND TaxYearKnown(p,y) -> ScopeReady(p)",
     "A payroll case needs an identified jurisdiction and tax year.",
     lambda f: [(("ScopeReady", p), [("JurisdictionKnown", p),
                                     ("TaxYearKnown", p, y)])
```

Appendix A Python implementation (continued)

```
            for p in cases(f) for y in [YEAR]
            if ("JurisdictionKnown", p) in f and ("TaxYearKnown", p, y) in f]),
    ("P2", "Employee(x) OR Contractor(x) -> ClassificationKnown(x)",
     "Worker classification must be identified.",
     lambda f: [(("ClassificationKnown", x), [fact])
                for fact in f if fact[0] in ("Employee", "Contractor")
                for x in [fact[1]]]),
    ("P3", "ScopeReady(p) AND ClassificationKnown(x) AND WorkerOnCase(x,p) AND InputsComplete(p) AND ProfileValid(p) -> CalculationReady(p)",
     "Calculation requires scope, classification, complete inputs and a valid profile.",
     lambda f: [(("CalculationReady", p), [("ScopeReady", p),
                    ("ClassificationKnown", x), ("WorkerOnCase", x, p),
                    ("InputsComplete", p), ("ProfileValid", p)])
                for p in cases(f) for x in worker_for_case(f, p)
                if ("ScopeReady", p) in f and ("ClassificationKnown", x) in f
                and ("WorkerOnCase", x, p) in f and ("InputsComplete", p) in f
                and ("ProfileValid", p) in f]),
    ("P4", "CalculationReady(p) AND TaxableBaseReady(p) -> TaxBaseValidated(p)",
     "The configured taxable earnings/benefit base must be ready before component assessment.",
     lambda f: [(("TaxBaseValidated", p), [("CalculationReady", p),
                    ("TaxableBaseReady", p)]) for p in cases(f)
                if ("CalculationReady", p) in f and ("TaxableBaseReady", p) in f]),
    ("P5", "USCase(p) AND Employee(x) AND WorkerOnCase(x,p) AND TaxBaseValidated(p) -> FederalWithholding(p)",
     "US employee cases enter the federal income-tax withholding path.",
     lambda f: [(("FederalWithholding", p), [("USCase", p), ("Employee", x),
                    ("WorkerOnCase", x, p), ("TaxBaseValidated", p)])
                for p in cases(f) for x in worker_for_case(f,p)
                if all(z in f for z in [("USCase",p),("Employee",x),
                    ("WorkerOnCase",x,p),("TaxBaseValidated",p)])]),
    ("P6", "USCase(p) AND Employee(x) AND WorkerOnCase(x,p) AND TaxBaseValidated(p) -> SocialSecurityAssessment(p)",
     "Eligible US employee cases enter Social Security assessment, subject to annual wage-base/YTD rules.",
     lambda f: [(("SocialSecurityAssessment", p), [("USCase", p), ("Employee", x),
                    ("WorkerOnCase", x, p), ("TaxBaseValidated", p)])
                for p in cases(f) for x in worker_for_case(f,p)
                if all(z in f for z in [("USCase",p),("Employee",x),
                    ("WorkerOnCase",x,p),("TaxBaseValidated",p)])]),
    ("P7", "USCase(p) AND Employee(x) AND WorkerOnCase(x,p) AND TaxBaseValidated(p) -> MedicareAssessment(p)",
     "Eligible US employee cases enter Medicare assessment, including additional Medicare checks where applicable.",
     lambda f: [(("MedicareAssessment", p), [("USCase", p), ("Employee", x),
                    ("WorkerOnCase", x, p), ("TaxBaseValidated", p)])
                for p in cases(f) for x in worker_for_case(f,p)
                if all(z in f for z in [("USCase",p),("Employee",x),
                    ("WorkerOnCase",x,p),("TaxBaseValidated",p)])]),
    ("P8", "USCase(p) AND Employee(x) AND WorkerOnCase(x,p) AND StateLocalObligation(p) -> StateLocalTaxPath(p)",
     "State/local tax is assessed when the work/residence facts establish an obligation.",
     lambda f: [(("StateLocalTaxPath", p), [("USCase", p), ("Employee", x),
                    ("WorkerOnCase", x, p), ("StateLocalObligation", p)])
                for p in cases(f) for x in worker_for_case(f,p)
                if all(z in f for z in [("USCase",p),("Employee",x),
                    ("WorkerOnCase",x,p),("StateLocalObligation",p)])]),
    ("P9", "CanadaCase(p) AND Employee(x) AND WorkerOnCase(x,p) AND TaxBaseValidated(p) -> CanadianIncomeTaxPath(p)",
     "Canadian employee cases enter federal and applicable provincial/territorial income-tax withholding.",
     lambda f: [(("CanadianIncomeTaxPath", p), [("CanadaCase", p), ("Employee", x),
                    ("WorkerOnCase", x, p), ("TaxBaseValidated", p)])
                for p in cases(f) for x in worker_for_case(f,p)
                if all(z in f for z in [("CanadaCase",p),("Employee",x),
                    ("WorkerOnCase",x,p),("TaxBaseValidated",p)])]),
    ("P10", "CanadaCase(p) AND Employee(x) AND WorkerOnCase(x,p) AND TaxBaseValidated(p) -> CPPQPPAssessment(p) AND EIAssessment(p)",
     "Canadian employee cases enter CPP/QPP and EI assessment, subject to applicable exemptions and annual limits.",
     lambda f: [((pred, p), [("CanadaCase", p), ("Employee", x),
                    ("WorkerOnCase", x, p), ("TaxBaseValidated", p)])
                for p in cases(f) for x in worker_for_case(f,p)
                if all(z in f for z in [("CanadaCase",p),("Employee",x),
                    ("WorkerOnCase",x,p),("TaxBaseValidated",p)])
                for pred in ("CPPQPPAssessment","EIAssessment")]),
    ("P11", "CalculationReady(p) AND TaxCalculated(p) AND ValidationPassed(p) -> Finalised(p)",
     "Payroll is finalised only after calculation and validation pass.",
     lambda f: [(("Finalised", p), [("CalculationReady", p), ("TaxCalculated", p),
                    ("ValidationPassed", p)]) for p in cases(f)
                if all(z in f for z in [("CalculationReady",p),("TaxCalculated",p),
                    ("ValidationPassed",p)])]),
    ("P12", "USCase(p) AND Employee(x) AND WorkerOnCase(x,p) AND Finalised(p) AND ReportableWages(x,y) AND YearEndReady(x,y) -> W2Required(x,y)",
     "A reportable US employee is routed to W-2 generation after payroll/year-end readiness.",
     lambda f: [(("W2Required", x, y), [("USCase",p),("Employee",x),
                    ("WorkerOnCase",x,p),("Finalised",p),("ReportableWages",x,y),
                    ("YearEndReady",x,y)])
                for p in cases(f) for x in worker_for_case(f,p) for y in [YEAR]
                if all(z in f for z in [("USCase",p),("Employee",x),
                    ("WorkerOnCase",x,p),("Finalised",p),("ReportableWages",x,y),
                    ("YearEndReady",x,y)])]),
    ("P13", "USCase(p) AND Contractor(x) AND WorkerOnCase(x,p) AND Finalised(p) AND ReportableContractorPay(x,y) AND YearEndReady(x,y) -> Form1099Review(x,y)",
```

Appendix A Python implementation (continued)

```
     "Reportable US contractor payments enter the appropriate 1099 type/obligation review.",
     lambda f: [(("Form1099Review", x, y), [("USCase",p),("Contractor",x),
                    ("WorkerOnCase",x,p),("Finalised",p),("ReportableContractorPay",x,y),
                    ("YearEndReady",x,y)])
                for p in cases(f) for x in worker_for_case(f,p) for y in [YEAR]
                if all(z in f for z in [("USCase",p),("Contractor",x),
                    ("WorkerOnCase",x,p),("Finalised",p),("ReportableContractorPay",x,y),
                    ("YearEndReady",x,y)])]),
    ("P14", "CanadaCase(p) AND Employee(x) AND WorkerOnCase(x,p) AND Finalised(p) AND ReportableEmploymentIncome(x,y) AND YearEndReady(x,y) -> T4Required(x,y)",
     "A reportable Canadian employee is routed to T4 generation.",
     lambda f: [(("T4Required", x, y), [("CanadaCase",p),("Employee",x),
                    ("WorkerOnCase",x,p),("Finalised",p),("ReportableEmploymentIncome",x,y),
                    ("YearEndReady",x,y)])
                for p in cases(f) for x in worker_for_case(f,p) for y in [YEAR]
                if all(z in f for z in [("CanadaCase",p),("Employee",x),
                    ("WorkerOnCase",x,p),("Finalised",p),("ReportableEmploymentIncome",x,y),
                    ("YearEndReady",x,y)])]),
    ("P15", "FormIssued(f,x,y) AND AdjustmentRecorded(a,x,y) AND FormDataChanged(a,x,y) -> CorrectedFormRequired(f,x,y)",
     "A post-issuance adjustment affecting annual form data initiates a correction workflow.",
     lambda f: [(("CorrectedFormRequired", form, x, y), [
                    ("FormIssued",form,x,y),("AdjustmentRecorded",a,x,y),
                    ("FormDataChanged",a,x,y)])
                for form,x,y in facts_by_pred(f,"FormIssued")
                for a in [z[1] for z in f if z[0]=="AdjustmentRecorded"
                          and z[2]==x and z[3]==y]
                if ("AdjustmentRecorded",a,x,y) in f and ("FormDataChanged",a,x,y) in f]),
]

def cases(f):
    return sorted({z[1] for z in f if z[0] in ("USCase","CanadaCase")})

def worker_for_case(f, p):
    return [z[1] for z in f if z[0]=="WorkerOnCase" and z[2]==p]

def facts_by_pred(f, pred):
    return [z[1:] for z in f if z[0]==pred]

def fmt(f):
    return f[0] + "(" + ", ".join(map(str, f[1:])) + ")"

def forward_chain(base):
    kb = set(base)
    proof = {}
    changed = True
    while changed:
        changed = False
        for name, _, _, fn in RULES:
            for fact, premises in fn(kb):
                if fact not in kb:
                    kb.add(fact)
                    proof[fact] = (name, premises)
                    changed = True
    return kb, proof

def backward_trace(goal, kb, proof, depth=0, seen=None):
    if seen is None:
        seen = set()
    pad = "  " * depth
    if goal in seen:
        print(pad + fmt(goal) + " [cycle prevented]")
        return
    seen.add(goal)
    if goal in proof:
        rule, premises = proof[goal]
        print(pad + fmt(goal) + " <- derived by " + rule)
        for premise in premises:
            backward_trace(premise, kb, proof, depth+1, seen.copy())
    elif goal in kb:
        print(pad + fmt(goal) + " <- given FACT")
    else:
        print(pad + fmt(goal) + " <- NOT PROVED")

def main():
    kb, proof = forward_chain(FACTS)
    print("AIMLZG557 - CROSS-BORDER PAYROLL KNOWLEDGE BASE")
    print("Synthetic cases | Tax year:", YEAR)
    print("Base facts:", len(FACTS), "| Inference rules:", len(RULES),
          "| Derived facts:", len(proof))
    print("Forward chaining completed until no new facts were derived.")
    print("\nQUERY 1: Does US employee U101 enter the W-2 route, and which tax paths apply?")
```

Appendix A Python implementation (continued)

```
    for goal in [("FederalWithholding","P1"), ("SocialSecurityAssessment","P1"),
                 ("MedicareAssessment","P1"), ("StateLocalTaxPath","P1"),
                 ("Finalised","P1"), ("W2Required","U101",YEAR)]:
        print(("PROVED: " if goal in kb else "NOT PROVED: ") + fmt(goal))
    print("\nQUERY 2: Does US contractor C301 enter the 1099 review path?")
    goal = ("Form1099Review","C301",YEAR)
    print("Method: backward-chaining proof trace from the goal.")
    print("Goal:", fmt(goal))
    backward_trace(goal, kb, proof)
    print("ANSWER:", "YES - route to 1099 type/reporting review." if goal in kb
          else "NO - route not established.")
    print("\nQUERY 3: Is Canadian employee U201 ready for T4, and which tax paths apply?")
    for goal in [("CanadianIncomeTaxPath","P5"), ("CPPQPPAssessment","P5"),
                 ("EIAssessment","P5"), ("Finalised","P5"),
                 ("T4Required","U201",YEAR)]:
        print(("PROVED: " if goal in kb else "NOT PROVED: ") + fmt(goal))
    print("\nQUERY 4: Can Canadian payroll case P6 for U202 be finalised?")
    goal = ("Finalised","P6")
    print("Method: backward-chaining proof trace from the goal.")
    backward_trace(goal, kb, proof)
    print("ValidationFailed(P6) is an explicit base fact; ValidationPassed(P6) is absent.")
    print("ANSWER: NO - do not finalise; send to payroll exception review.")
    print("\nQUERY 5: Is a corrected W-2 workflow required for U102?")
    goal = ("CorrectedFormRequired","W2","U102",YEAR)
    backward_trace(goal, kb, proof)
    print("ANSWER:", "YES - initiate the applicable corrected-form workflow."
          if goal in kb else "NO - correction not established.")
    print("\nSUMMARY")
    for label, goal in [
        ("U101 W-2",("W2Required","U101",YEAR)),
        ("C301 1099 review",("Form1099Review","C301",YEAR)),
        ("U201 T4",("T4Required","U201",YEAR)),
        ("U202 finalised",("Finalised","P6")),
        ("U102 corrected W-2",("CorrectedFormRequired","W2","U102",YEAR)),
    ]:
        print(f"{label:24} {'PROVED' if goal in kb else 'NOT PROVED'}")
    print("\nEXECUTION COMPLETE")

if __name__ == "__main__":
    main()
```

Appendix A Execution evidence

W-2 query and the backward-chaining 1099 proof trace.

```
/Users/adityasaraswat/PycharmProjects/PythonProject/.venv/bin/python /Users/adityasaraswat/PycharmProjects/PythonProject/payroll_kb.py
AIMLZG557 - CROSS-BORDER PAYROLL KNOWLEDGE BASE
Synthetic cases | Tax year: 2026
Base facts: 80 | Inference rules: 15 | Derived facts: 45
Forward chaining completed until no new facts were derived.

QUERY 1: Does US employee U101 enter the W-2 route, and which tax paths apply?
PROVED: FederalWithholding(P1)
PROVED: SocialSecurityAssessment(P1)
PROVED: MedicareAssessment(P1)
PROVED: StateLocalTaxPath(P1)
PROVED: Finalised(P1)
PROVED: W2Required(U101, 2026)

QUERY 2: Does US contractor C301 enter the 1099 review path?
Method: backward-chaining proof trace from the goal.
Goal: Form1099Review(C301, 2026)
Form1099Review(C301, 2026) <- derived by P13
  USCase(P3) <- given FACT
  Contractor(C301) <- given FACT
  WorkerOnCase(C301, P3) <- given FACT
  Finalised(P3) <- derived by P11
    CalculationReady(P3) <- derived by P3
      ScopeReady(P3) <- derived by P1
        JurisdictionKnown(P3) <- given FACT
        TaxYearKnown(P3, 2026) <- given FACT
      ClassificationKnown(C301) <- derived by P2
        Contractor(C301) <- given FACT
      WorkerOnCase(C301, P3) <- given FACT
      InputsComplete(P3) <- given FACT
      ProfileValid(P3) <- given FACT
    TaxCalculated(P3) <- given FACT
    ValidationPassed(P3) <- given FACT
  ReportableContractorPay(C301, 2026) <- given FACT
  YearEndReady(C301, 2026) <- given FACT
ANSWER: YES - route to 1099 type/reporting review.
```

summary and successful completion (exit code 0).

```
QUERY 3: Is Canadian employee U201 ready for T4, and which tax paths apply?
PROVED: CanadianIncomeTaxPath(P5)
PROVED: CPPQPPAssessment(P5)
PROVED: EIAssessment(P5)
PROVED: Finalised(P5)
PROVED: T4Required(U201, 2026)

QUERY 4: Can Canadian payroll case P6 for U202 be finalised?
Method: backward-chaining proof trace from the goal.
Finalised(P6) <- NOT PROVED
ValidationFailed(P6) is an explicit base fact; ValidationPassed(P6) is absent.
ANSWER: NO - do not finalise; send to payroll exception review.

QUERY 5: Is a corrected W-2 workflow required for U102?
CorrectedFormRequired(W2, U102, 2026) <- derived by P15
  FormIssued(W2, U102, 2026) <- given FACT
  AdjustmentRecorded(A1, U102, 2026) <- given FACT
  FormDataChanged(A1, U102, 2026) <- given FACT
ANSWER: YES - initiate the applicable corrected-form workflow.

SUMMARY
U101 W-2                PROVED
C301 1099 review        PROVED
U201 T4                 PROVED
U202 finalised          NOT PROVED
U102 corrected W-2      PROVED

EXECUTION COMPLETE

Process finished with exit code 0
```